

[illegible]

[illegible]

```
create_file() {
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE}   ${NC}"
echo -e "${BLUE}===== ${NC}"
# 1
echo -e "${YELLOW}---   ---${NC}"
read -p " ( ): " target_dir
if [ -z "$target_dir" ]; then
target_dir=$(pwd)
fi
# 
if [ ! -d "$target_dir" ]; then
echo -e "${RED}[ ] ' $target_dir'   ${NC}"
return 1
fi
# 2
read -p ": " filename
# 
if [ -z "$filename" ]; then
echo -e "${RED}[ ]   ${NC}"
return 1
fi
# 
file_path="$target_dir/$filename"
# 
if [ -f "$file_path" ]; then
echo -e "${YELLOW}[ ] ' $file_path'   ${NC}"
read -p "(y/n): " overwrite
if [ "$overwrite" != "y" ] && [ "$overwrite" != "Y" ]; then
echo -e "${YELLOW}   ${NC}"
return 0
fi
fi
# 
echo -e "${YELLOW}[...]... ${NC}"
touch "$file_path"
if [ $? -eq 0 ]; then
# 644rw-r--r--
chmod 644 "$file_path"
echo -e "${GREEN}[ ] ' $file_path'   ${NC}"
# 
read -p "(y/n): " write_content
if [ "$write_content" = "y" ] || [ "$write_content" = "Y" ]; then
echo -e "${YELLOW} :wq   ${NC}"
echo "---   ---"
# 
> "$file_path" # 
while IFS= read -r line; do
if [ "$line" = ":wq" ]; then
break
fi
echo "$line" >> "$file_path"
done
echo "---   ---"
echo -e "${GREEN}:wq   ${NC}"
fi
# 
echo -e "\n${YELLOW}---   ---${NC}"
ls -lh "$file_path"
echo " : $(stat -c%s "$file_path")"
echo " : $(stat -c%y "$file_path")"
log_operation "" "$file_path" ""
else
echo -e "${RED}[ ]   ${NC}"
log_operation "" "$file_path" ""
return 1
fi
}
# =====
# 4/
# 
# 1. 
# 2. 
# 3.
```

```
# 4. ██████████
# =====
modify_file() {
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE} ██████████ ${NC}"
echo -e "${BLUE}===== ${NC}"
# ████████
read -p "██████████: " file_path
# ████████
if [ ! -f "$file_path" ]; then
echo -e "${RED}[██] ██ '$file_path' ██████ ${NC}"
return 1
fi
# ██████████
echo -e "\n${YELLOW}--- ████████ (██10██) --- ${NC}"
head -n 10 "$file_path"
echo "..."
echo -e "${YELLOW}██████: $(wc -l < "$file_path") █, $(stat -c%s "$file_path") █████ ${NC}"
echo ""
# ██████████
backup_path="$file_path.bak.$(date +%Y%m%d%H%M%S)"
cp "$file_path" "$backup_path"
echo -e "${GREEN}[██] ████████: $backup_path ${NC}"
# ████████
echo -e "${YELLOW}--- ████████ --- ${NC}"
echo "1. ██████████"
echo "2. ██████████"
echo "3. ██████████"
echo "4. ██████████"
read -p "████ (1-4): " mode
case $mode in
1)
# ████████ >> █████
echo -e "${YELLOW}██████████████████ :wq █████ ${NC}"
while IFS= read -r line; do
if [ "$line" = ":wq" ]; then
break
fi
echo "$line" >> "$file_path"
done
echo -e "${GREEN}[██] ██████████ ${NC}"
;;
2)
# ████████ > █████
echo -e "${YELLOW}██████████████████ :wq █████ ${NC}"
> "$file_path" # █████
while IFS= read -r line; do
if [ "$line" = ":wq" ]; then
break
fi
echo "$line" >> "$file_path"
done
echo -e "${GREEN}[██] ██████████ ${NC}"
;;
3)
# ████████
read -p "██████████: " line_num
echo -e "${YELLOW}██████████████████ :wq █████ ${NC}"
temp_file=$(mktemp)
line_count=1
inserted=0
while IFS= read -r line || [ -n "$line" ]; do
if [ "$line_count" = "$line_num" ] && [ "$inserted" -eq 0 ]; then
while IFS= read -r new_line; do
if [ "$new_line" = ":wq" ]; then
inserted=1
break
fi
echo "$new_line" >> "$temp_file"
done
fi
echo "$line" >> "$temp_file"
line_count=$((line_count + 1))
done < "$file_path"
```

```

mv "$temp_file" "$file_path"
echo -e "${GREEN}[██] ██████████ ${line_num} █████${NC}"
;;
4)
# ████████████████████sed███
read -p "██████████████████: " old_str
read -p "██████████████████: " new_str
# sed -i ██████████s/old/new/g ██████
sed -i "s/$old_str/$new_str/g" "$file_path"
echo -e "${GREEN}[██] ██ '$old_str' ██████ '$new_str'${NC}"
;;
*)
echo -e "${RED}[██] ████████${NC}"
;;
esac
log_operation "████" "$file_path" "██"
}
# =====
# ███5██████
# ████████
# 1. ██████████
# 2. ████████████████████
# 3. ██████████
# =====
delete_file() {
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE} ██████${NC}"
echo -e "${BLUE}===== ${NC}"
read -p "██████████████████: " file_path
# ████████████████████
if [ ! -f "$file_path" ]; then
echo -e "${RED}[██] ██ '$file_path' ████████████████████${NC}"
return 1
fi
# ████████
echo -e "${YELLOW}--- ██████████ ---${NC}"
ls -lh "$file_path"
# ████████████████████
system_dirs=("/etc/" "/usr/bin/" "/usr/sbin/")
for sys_dir in "${system_dirs[@]}"; do
if [ "$file_path" == $sys_dir* ]; then
echo -e "${RED}[██] ████████████████████${NC}"
break
fi
done
# ████████
echo -e "${RED}[██] ████████████████████${NC}"
read -p "██████████ '$file_path' ███(██ yes ██): " confirm
if [ "$confirm" != "yes" ]; then
echo -e "${YELLOW}██████████████████${NC}"
return 0
fi
# ████████
rm -f "$file_path"
if [ $? -eq 0 ] && [ ! -f "$file_path" ]; then
echo -e "${GREEN}[██] ██ '$file_path' ████████${NC}"
log_operation "████" "$file_path" "██"
else
echo -e "${RED}[██] ████████████████████${NC}"
log_operation "████" "$file_path" "██"
return 1
fi
}
# =====
# ████████████████████
# =====
browse_directory() {
echo -e "${BLUE}===== ${NC}"
echo -e "${BLUE} ████████████████████${NC}"
echo -e "${BLUE}===== ${NC}"
read -p "██████████████████ (██████████): " browse_path
if [ -z "$browse_path" ]; then
browse_path=$(pwd)
fi
}

```

[illegible]

```
;;  
esac  
echo ""  
read -p "■■■■■■..."  
done  
}  
# ■■■■  
main_menu
```